Can Wasm help us run fewer servers?

Sustainable computing · Java services vs Rust/Wasm on wasmCloud

A smaller memory footprint.
A larger CPU appetite.
A conditional case for consolidation.

Promising for memory-bound fleets; a production replacement is not yet proven.

The result is a trade-off, not a winner

The machine’s limiting resource determines which implementation fits better.

WASM’S STRONGEST RESULT

Less RAM

Repeated at idle and under load

JAVA’S STRONGEST RESULT

Less CPU

Lower typical and p95 latency

THE SUSTAINABILITY QUESTION

Fewer hosts?

Only if CPU and latency still fit

A runtime change is valuable when it lets us consolidate and retire capacity.

What we actually compared

Complete accepted-order HTTP request paths, not isolated language benchmarks.

JAVA

Four service containers
Orders · Inventory · Products · Notifications

RUST / WASM

wasmCloud host + native providers
Components share runtime resources

One 10-core Docker VM · application resource totals · shared dependencies excluded

How much evidence do we have?

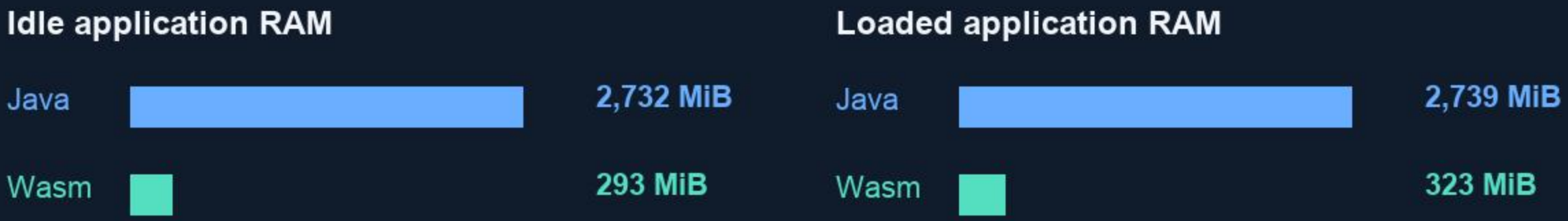
Historical troubleshooting was retained separately from frozen validation.

Study	Coverage	Question
Footprint	**5 alternating pairs**	**Idle + loaded resource use**
Memory density	**3 sweeps per version**	**Smallest passing allocation**
Cold / cached startup	**5 pairs per version**	**First real accepted order**
Capacity + stability	**3 pairs + 30 min each**	**Sustained load and limits**

Multiple comparisons support the trade-off; long-term production behavior remains open.

Idle memory is the clearest opportunity

Frozen footprint study · medians across five runs · 100 orders/s when loaded



88% less loaded RAM

A smaller resident footprint could reduce memory stranded between traffic bursts.

The CPU cost moves in the opposite direction

Same frozen footprint trials · 100 orders/s · both versions accepted all work

Loaded application CPU



Order p95 latency



Wasm: ~1.86× CPU

More memory capacity is useful only while CPU and response-time headroom remain.

Memory density: a strong, bounded result

25 orders/s per application · CPU headroom · p95 target below 100 ms

Smallest cap passing all three sweeps



8 GiB APPLICATION MEMORY POOL

4 vs 16

Java vs Wasm allocations

4× projected memory-only density

Projection, not sixteen tested concurrent applications. CPU and infrastructure still count.

The machine profile can reverse the answer

Illustrative packing ceilings from the 100/s resource profile, not deployment sizing.

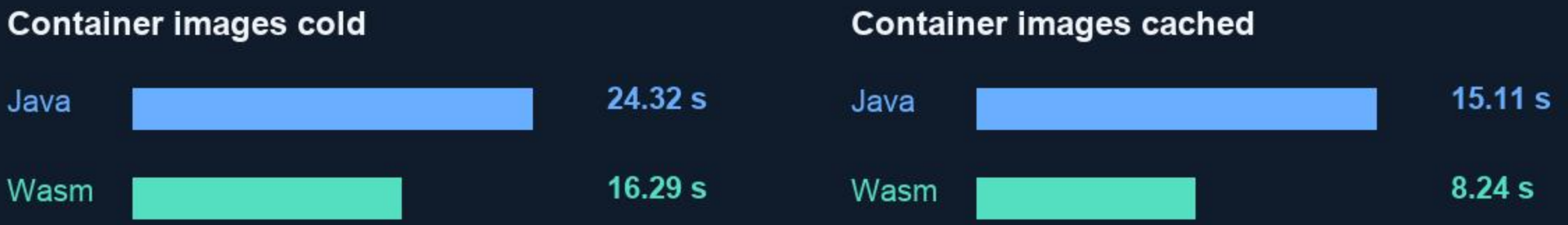
Usable application pool	Java copies	Wasm copies
4 GiB RAM / 4 CPU cores	1 · memory bound	7 · CPU bound
32 GiB RAM / 4 CPU cores	11 · memory bound	7 · CPU bound

copies ≤ min(RAM pool ÷ RAM per copy, CPU pool ÷ CPU per copy)

Wasm favors memory-tight, CPU-rich hosts; Java may fit CPU-tight hosts better.

Startup: Wasm served the first order sooner

Five paired starts per version · complete application scaled from zero · warm dependencies

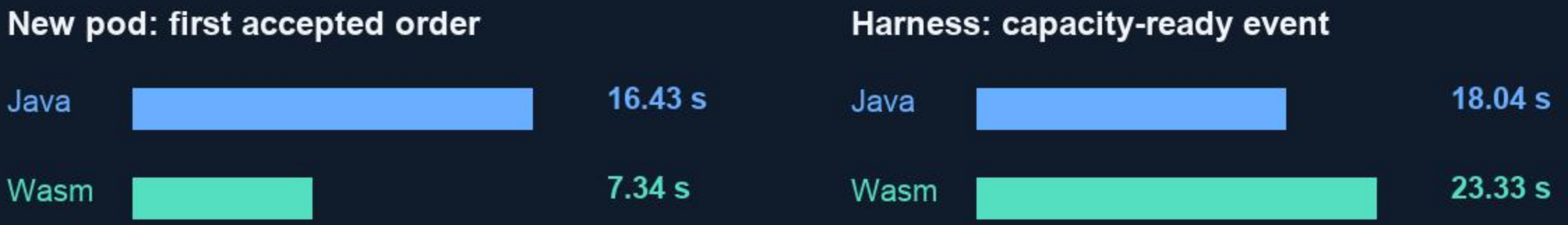


~33% sooner cold · ~45% sooner cached

Faster activation could reduce pre-warmed capacity—if the complete scaling loop works.

Faster startup did not guarantee faster scale-out

Earlier five-pair 400/s burst study · readiness definitions change the interpretation



Wasm starts quickly; serving capacity arrives through a wider control loop.

Do not promise fewer warm servers from pod startup time alone.

Tune allocation before declaring a capacity gap

Same total application budget: 2 CPU cores / 4 GiB RAM

Java: clean five-minute confirmed rate



ONLY CPU DISTRIBUTION CHANGED

+75%

Confirmed rate at the same budget

Right-sizing the existing Java stack is also a consolidation opportunity.

At sustained 350/s, both accepted every issued order

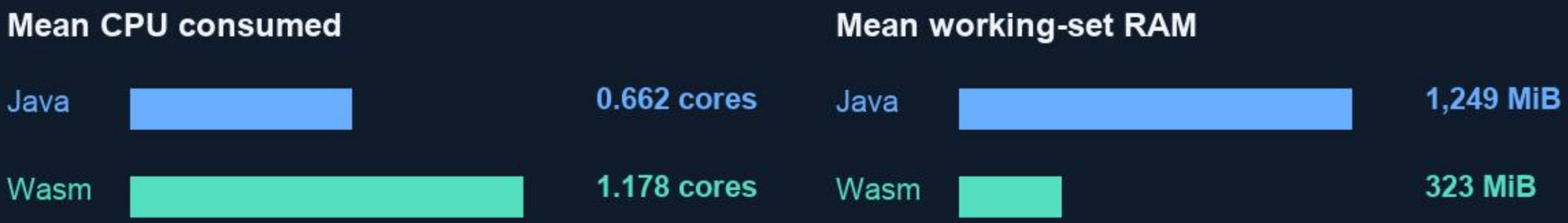
One fresh-pod 30-minute run each · tuned Java · identical 2-core / 4-GiB budgets

Metric	Java	Wasm
Accepted orders	629,954	630,001
HTTP / body failures	0	0
Unscheduled arrivals	47	0
p95 latency	21.77 ms	51.65 ms

Wasm passed the strict gate; Java missed zero-drop by 0.0075% of planned work.

Sustained load confirms the resource trade-off

30-minute matched 350/s runs · application CPU and working-set memory



Wasm: 1.78× CPU · 74% less average RAM

For consolidation, model both resources at the traffic each instance must handle.

The unresolved issue: Wasm memory kept growing

First vs last five-minute mean RAM in the 30-minute 350/s trial

Working-set memory	Java	Wasm
First five minutes	1,215 MiB	202 MiB
Last five minutes	1,288 MiB	445 MiB
Increase	+6%	+120%

No demonstrated plateau. Long-lived density requires a longer memory investigation.

Sustainability: what we can—and cannot—claim

Resource measurements support a hypothesis; they do not measure its environmental outcome.

SUPPORTED BY THE TESTS

Less application RAM
Faster first accepted order
More loaded CPU
Usually higher p95 latency

STILL TO BE DEMONSTRATED

Fewer physical servers
Lower kWh per accepted order
Lower lifecycle carbon
Stable long-term packed density

CPU-seconds and MiB are useful indicators. Neither is a carbon measurement.

A practical replacement decision

Choose by the bottleneck and service target, rather than runtime preference.

Deployment profile	Assessment	Reason
Memory tight; CPU available	**Pilot Wasm**	**Strongest density opportunity**
CPU tight; latency critical	**Favor current Java**	**Lower CPU and p95 here**
Bursty; much idle capacity	**Pilot Wasm**	**Smaller idle footprint; fast start**
Long-lived, high throughput	**Investigate first**	**Memory plateau unresolved**

A targeted pilot is supported; a blanket migration claim is not.

The next proof should be fewer real machines

A production-shaped consolidation trial, rather than another isolated latency benchmark.

1 Establish stability

Explain Wasm memory growth;
confirm its steady-state envelope.

2 Pack real workloads

Same SLA and demand, full platform
overhead, measured CPU headroom.

3 Measure the outcome

Hosts needed, accepted orders,
latency, and whole-host kWh.

Success = fewer hosts for the same useful work and SLA, with lower measured energy.

Evidence and reproducibility

Keep the conclusion attached to the tested system and the retained raw results.

Frozen validation

monitoring/validation/RESULTS.md
.results/20261106T204859Z/analysis.html

Allocation, repeated capacity and long-run validation

monitoring/capacity/VALIDATION_RESULTS.md
.results/validation-20261107T233519Z/analysis.html

Wasm is a credible consolidation candidate where memory—not CPU—is the constraint.